Assignment: SMS Spam vs. Ham Classification Using a Convolutional Neural Network (CNN)

Deep Learning / Natural Language Processing Course

Objective

Build, train, and evaluate a **Convolutional Neural Network (CNN)** using PyTorch to classify SMS messages as **spam** or **ham** (legitimate). The pipeline covers:

1. Data ingestion and preprocessing (tokenization, vocabulary, padding/truncating to `MAX_LEN`)
2. Custom PyTorch `Dataset` and `DataLoader` for mini-batch training
3. A **Text CNN** with embedding, multiple 1D convolution filter sizes, max-pooling, and dropout
4. Training with `BCEWithLogitsLoss` and Adam optimizer
5. Evaluation with accuracy, confusion matrices, and classification reports
6. Inference function to classify custom text messages with confidence scores

Dataset

SMS Spam Collection — loaded from a public GitHub CSV with `encoding='latin-1'`.

---

1. Imports and Setup

In [1]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from collections import Counter
import string
import random

%matplotlib inline
```

```
plt.rcParams['figure.figsize'] = (10, 5)
plt.rcParams['figure.dpi'] = 100

# Reproducibility
SEED = 42
torch.manual_seed(SEED)
random.seed(SEED)
np.random.seed(SEED)

print(f"PyTorch version: {torch.__version__}")
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Device: {DEVICE}")
```

```
PyTorch version: 2.14.1
Device: cpu
```

---

2. Data Loading and Exploratory Data Analysis (EDA)

We load the raw dataset using `pd.read_csv()` with `encoding='latin-1'` to handle special characters, then explore its structure and class distribution.

In [2]:
```
# Load raw dataset
url = ("https://raw.githubusercontent.com/mohitgupta-101/"
       "Kaggle-SMS-Spam-Collection-Dataset-/master/spam.csv")
df = pd.read_csv(url, encoding='latin-1')

# The CSV has extra unnamed columns — keep only the first two
df = df[['v1', 'v2']].rename(columns={'v1': 'label', 'v2': 'message'})

print(f"Dataset shape: {df.shape}")
print(f"\nFirst 5 rows:")
df.head()
```

```
Dataset shape: (5572, 2)

First 5 rows:
```

Out[2]:

	label	message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

In [3]:
```
# Class distribution
print("Class Distribution:")
print(df['label'].value_counts())
print(f"\nSpam ratio: {(df['label'] == 'spam').mean():.2%}")
print(f"Ham ratio:  {(df['label'] == 'ham').mean():.2%}")
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Bar chart of class counts
counts = df['label'].value_counts()
colors = ['#2ecc71', '#e74c3c']
axes[0].bar(counts.index, counts.values, color=colors)
axes[0].set_title('Class Distribution')
axes[0].set_ylabel('Count')
for i, (label, count) in enumerate(zip(counts.index, counts.values)):
    axes[0].text(i, count + 50, str(count), ha='center', fontweight='bold')

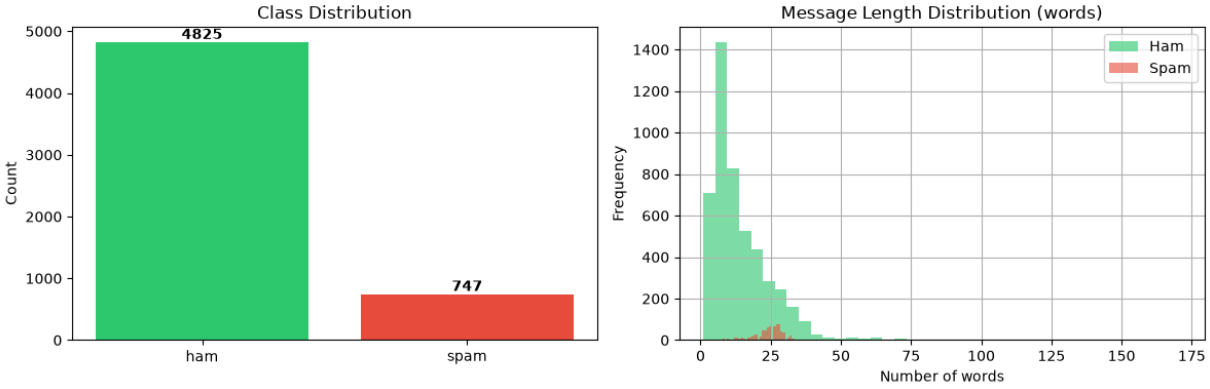
# Message length distribution
df['msg_length'] = df['message'].apply(lambda x: len(x.split()))
df[df['label'] == 'ham']['msg_length'].hist(
    bins=40, alpha=0.6, label='Ham', color='#2ecc71', ax=axes[1])
df[df['label'] == 'spam']['msg_length'].hist(
    bins=40, alpha=0.6, label='Spam', color='#e74c3c', ax=axes[1])
axes[1].set_title('Message Length Distribution (words)')
axes[1].set_xlabel('Number of words')
axes[1].set_ylabel('Frequency')
axes[1].legend()

plt.tight_layout()
plt.show()

print(f"\nAvg message length (ham):  {df[df['label']=='ham']['msg_length'].m
print(f"Avg message length (spam): {df[df['label']=='spam']['msg_length'].me
print(f"Max message length:        {df['msg_length'].max()} words")
```

```
Class Distribution:
label
ham     4825
spam     747
Name: count, dtype: int64

Spam ratio: 13.41%
Ham ratio:  86.59%
```




```
Avg message length (ham):  14.2 words
Avg message length (spam): 23.9 words
Max message length:        171 words
```

In [4]:
```
# Sample messages
print("=" * 60)
```

```
print("SAMPLE HAM MESSAGES:")
print("=" * 60)
for msg in df[df['label'] == 'ham']['message'].sample(3, random_state=42).va
    print(f"  > {msg[:100]}")

print(f"\n{'=' * 60}")
print("SAMPLE SPAM MESSAGES:")
print("=" * 60)
for msg in df[df['label'] == 'spam']['message'].sample(3, random_state=42).v
    print(f"  > {msg[:100]}")
```

```
============================================================
SAMPLE HAM MESSAGES:
============================================================
  > I am late,so call you tomorrow morning.take care sweet dreams....u and m
e...ummifying...bye.
  > U r too much close to my heart. If u go away i will be shattered. Plz st
ay with me.
  > Wait  &lt;#&gt;  min..

============================================================
SAMPLE SPAM MESSAGES:
============================================================
  > Summers finally here! Fancy a chat or flirt with sexy singles in yr are
a? To get MATCHED up just rep
  > This is the 2nd time we have tried 2 contact u. U have won the 750 Pound
prize. 2 claim is easy, cal
  > Get ur 1st RINGTONE FREE NOW! Reply to this msg with TONE. Gr8 TOP 20 to
nes to your phone every week
```

---

3. Text Preprocessing and Train/Test Split

Preprocessing steps:

1. Map labels: `ham -> 0`, `spam -> 1`
2. Tokenize by splitting on whitespace and lowercasing
3. Split into **80% training** and **20% test** sets
4. Build vocabulary from **training data only** (with `<pad>` and `<unk>` tokens)
5. Pad/truncate all sequences to a fixed `MAX_LEN`

In [5]:
```
# Map labels to integers
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

# Tokenize
def tokenize(text):
    return text.lower().split()

df['tokens'] = df['message'].apply(tokenize)

# Train/test split (80/20)
train_df, test_df = train_test_split(df, test_size=0.2, random_state=SEED, s
```

```
train_df = train_df.reset_index(drop=True)
test_df = test_df.reset_index(drop=True)

print(f"Training samples: {len(train_df)}")
print(f"Test samples:     {len(test_df)}")
print(f"Train spam ratio: {train_df['label_num'].mean():.2%}")
print(f"Test spam ratio:  {test_df['label_num'].mean():.2%}")
```

```
Training samples: 4457
Test samples:     1115
Train spam ratio: 13.42%
Test spam ratio:  13.36%
```

In [6]:
```
# Build vocabulary from training data only
PAD_TOKEN = '<pad>'
UNK_TOKEN = '<unk>'

word_counts = Counter()
for tokens in train_df['tokens']:
    word_counts.update(tokens)

word2idx = {PAD_TOKEN: 0, UNK_TOKEN: 1}
for word, _ in word_counts.most_common():
    word2idx[word] = len(word2idx)

vocab_size = len(word2idx)
PAD_IDX = word2idx[PAD_TOKEN]
UNK_IDX = word2idx[UNK_TOKEN]

print(f"Vocabulary size: {vocab_size}")
print(f"PAD index: {PAD_IDX}")
print(f"UNK index: {UNK_IDX}")

# Count UNK tokens in test set
total_test_tokens = 0
unk_count = 0
for tokens in test_df['tokens']:
    for t in tokens:
        total_test_tokens += 1
        if t not in word2idx:
            unk_count += 1
print(f"Test tokens mapped to <unk>: {unk_count}/{total_test_tokens} ({unk_c
```

```
Vocabulary size: 11805
PAD index: 0
UNK index: 1
Test tokens mapped to <unk>: 1816/17468 (10.40%)
```

In [7]:
```
# Top 20 most frequent words in training data
top_20 = word_counts.most_common(20)
words, freqs = zip(*top_20)

plt.figure(figsize=(12, 4))
plt.bar(range(len(words)), freqs, color='steelblue')
plt.xticks(range(len(words)), words, rotation=45, ha='right')
plt.title('Top 20 Most Frequent Words in Training Data')
plt.ylabel('Frequency')
```

```
plt.tight_layout()
plt.show()
```



In [8]:
```
# Define MAX_LEN and encode/pad/truncate functions
MAX_LEN = 50

def encode_and_pad(tokens, word2idx, max_len):
    """Convert tokens to indices, then pad or truncate to max_len."""
    indices = [word2idx.get(t, UNK_IDX) for t in tokens]
    # Truncate if longer
    if len(indices) > max_len:
        indices = indices[:max_len]
    # Pad if shorter
    else:
        indices = indices + [PAD_IDX] * (max_len - len(indices))
    return indices

# Demonstrate
sample_tokens = train_df['tokens'].iloc[0]
sample_encoded = encode_and_pad(sample_tokens, word2idx, MAX_LEN)
print(f"MAX_LEN: {MAX_LEN}")
print(f"\nSample tokens (first 8): {sample_tokens[:8]}")
print(f"Encoded (first 8):       {sample_encoded[:8]}")
print(f"Encoded length:          {len(sample_encoded)}")

# Distribution of message lengths vs MAX_LEN
lengths = df['tokens'].apply(len)
print(f"\nMessages <= MAX_LEN ({MAX_LEN}): {(lengths <= MAX_LEN).mean():.1%}
print(f"Messages > MAX_LEN:  {(lengths > MAX_LEN).mean():.1%} (will be trunc
```

```
MAX_LEN: 50

Sample tokens (first 8): ['going', 'on', 'nothing', 'great.bye']
Encoded (first 8):       [73, 18, 351, 4644, 0, 0, 0, 0]
Encoded length:          50

Messages <= MAX_LEN (50): 98.9%
Messages > MAX_LEN:  1.1% (will be truncated)
```

---

4. Custom PyTorch Dataset and DataLoader

We create a custom `Dataset` class that stores encoded/padded sequences and labels, then wrap it with `DataLoader` for mini-batch training and shuffling.

In [9]:
```
class SpamDataset(Dataset):
    def __init__(self, dataframe, word2idx, max_len):
        self.labels = dataframe['label_num'].values
        self.texts = [
            encode_and_pad(tokens, word2idx, max_len)
            for tokens in dataframe['tokens']
        ]

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return (
            torch.tensor(self.texts[idx], dtype=torch.long),
            torch.tensor(self.labels[idx], dtype=torch.float)
        )


BATCH_SIZE = 64

train_dataset = SpamDataset(train_df, word2idx, MAX_LEN)
test_dataset = SpamDataset(test_df, word2idx, MAX_LEN)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f"Train batches: {len(train_loader)}")
print(f"Test batches:  {len(test_loader)}")

# Verify a batch
sample_x, sample_y = next(iter(train_loader))
print(f"\nSample batch — X shape: {sample_x.shape}, Y shape: {sample_y.shape
print(f"Sample labels: {sample_y[:8].tolist()}")
```

```
Train batches: 70
Test batches:  18

Sample batch — X shape: torch.Size([64, 50]), Y shape: torch.Size([64])
Sample labels: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

---

5. CNN Model Architecture

Architecture Diagram

```
   Input (token indices)          [batch_size, MAX_LEN]
           |
     Embedding Layer               [batch_size, MAX_LEN,
   embedding_dim]
```

```
          |
    Permute (0, 2, 1)              [batch_size, embedding_dim,
  MAX_LEN]
          |
    Conv1d (kernel=3) ----\
    Conv1d (kernel=4) -----+----> ReLU + MaxPool1d over seq
  [batch, n_filters * 3]
    Conv1d (kernel=5) ----/
          |
    Concatenate all filter outputs
          |
    Dropout (0.5)
          |
    Linear (n_filters * 3 -> 1)    [batch_size, 1]
          |
    Output (raw logit)
```

Key Design Points

- **Multiple kernel sizes** [3, 4, 5] capture n-gram patterns of different lengths (trigrams, 4-grams, 5-grams)
- **`nn.Conv1d`** expects `[batch, channels, seq_len]`, so we `permute(0, 2, 1)` after embedding
- **Max-pooling** over the full sequence extracts the strongest feature from each filter
- **Dropout** (0.5) regularizes before the final classification layer
- **Single output neuron** with `BCEWithLogitsLoss` for binary classification

In [10]:
```
class SpamCNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, n_filters, filter_sizes,
                 dropout_rate, pad_idx):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx

        # Multiple Conv1d layers with different kernel sizes
        self.convs = nn.ModuleList([
            nn.Conv1d(in_channels=embedding_dim,
                      out_channels=n_filters,
                      kernel_size=fs)
            for fs in filter_sizes
        ])

        self.dropout = nn.Dropout(dropout_rate)
        self.fc = nn.Linear(n_filters * len(filter_sizes), 1)

    def forward(self, text):
        # text: [batch_size, MAX_LEN]
        embedded = self.embedding(text)            # [batch, MAX_LEN, emb_di
        embedded = embedded.permute(0, 2, 1)       # [batch, emb_dim, MAX_LE

        # Apply each conv filter, then ReLU + global max pool
        conved = []
        for conv in self.convs:
```

```
            c = F.relu(conv(embedded))              # [batch, n_filters, L_c
            c = F.max_pool1d(c, c.shape[2])         # [batch, n_filters, 1]
            c = c.squeeze(2)                        # [batch, n_filters]
            conved.append(c)

        cat = torch.cat(conved, dim=1)             # [batch, n_filters * ler
        cat = self.dropout(cat)
        out = self.fc(cat)                          # [batch, 1]
        return out
```

In [11]:
```
# Hyperparameters
EMBEDDING_DIM = 100
N_FILTERS = 100
FILTER_SIZES = [3, 4, 5]
DROPOUT_RATE = 0.5
LEARNING_RATE = 0.001
NUM_EPOCHS = 10

model = SpamCNN(
    vocab_size=vocab_size,
    embedding_dim=EMBEDDING_DIM,
    n_filters=N_FILTERS,
    filter_sizes=FILTER_SIZES,
    dropout_rate=DROPOUT_RATE,
    pad_idx=PAD_IDX
).to(DEVICE)

print(model)

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_g
print(f"\nTotal parameters:     {total_params:,}")
print(f"Trainable parameters: {trainable_params:,}")
```

```
SpamCNN(
  (embedding): Embedding(11805, 100, padding_idx=0)
  (convs): ModuleList(
    (0): Conv1d(100, 100, kernel_size=(3,), stride=(1,))
    (1): Conv1d(100, 100, kernel_size=(4,), stride=(1,))
    (2): Conv1d(100, 100, kernel_size=(5,), stride=(1,))
  )
  (dropout): Dropout(p=0.5, inplace=False)
  (fc): Linear(in_features=300, out_features=1, bias=True)
)

Total parameters:     1,301,101
Trainable parameters: 1,301,101
```

In [12]:
```
# Verify forward pass with a sample batch
model.eval()
with torch.no_grad():
    sample_out = model(sample_x.to(DEVICE))

print("Forward pass verification:")
print(f"  Input shape:  {sample_x.shape}  (batch_size, MAX_LEN)")
print(f"  Output shape: {sample_out.shape} (batch_size, 1)")
```

```
print(f"  Sample logits (first 5): {sample_out[:5].squeeze().tolist()}")
probs = torch.sigmoid(sample_out[:5].squeeze())
print(f"  Sample probs  (first 5): {[round(p, 4) for p in probs.tolist()]}")
```

```
Forward pass verification:
  Input shape:  torch.Size([64, 50])  (batch_size, MAX_LEN)
  Output shape: torch.Size([64, 1]) (batch_size, 1)
  Sample logits (first 5): [-0.8932352066040039, -1.6888179779052734, -1.087
2348546981812, -0.6080661416053772, -1.4733742475509644]
  Sample probs  (first 5): [0.2904, 0.1559, 0.2521, 0.3525, 0.1864]
```

---

6. Training and Evaluation Functions

Training loop mechanics (per batch):

1. `optimizer.zero_grad()` — clear old gradients
2. `loss = criterion(predictions, labels)` — compute `BCEWithLogitsLoss`
3. `loss.backward()` — backpropagation
4. `optimizer.step()` — update weights

Evaluation:

- Toggle `model.eval()` and use `torch.no_grad()`
- Convert logits to predictions via `torch.sigmoid()` then threshold at 0.5

In [13]:
```
def train_one_epoch(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0.0
    correct = 0
    total = 0

    for texts, labels in loader:
        texts, labels = texts.to(device), labels.to(device)

        optimizer.zero_grad()
        logits = model(texts).squeeze(1)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * len(labels)
        preds = torch.round(torch.sigmoid(logits))
        correct += (preds == labels).sum().item()
        total += len(labels)

    return total_loss / total, correct / total


def evaluate(model, loader, criterion, device):
    model.eval()
```

```
    total_loss = 0.0
    correct = 0
    total = 0
    all_preds = []
    all_labels = []

    with torch.no_grad():
        for texts, labels in loader:
            texts, labels = texts.to(device), labels.to(device)

            logits = model(texts).squeeze(1)
            loss = criterion(logits, labels)

            total_loss += loss.item() * len(labels)
            preds = torch.round(torch.sigmoid(logits))
            correct += (preds == labels).sum().item()
            total += len(labels)

            all_preds.extend(preds.cpu().tolist())
            all_labels.extend(labels.cpu().tolist())

    return total_loss / total, correct / total, all_preds, all_labels
```

---

7. Model Training

We train for 10 epochs, tracking both training and validation loss/accuracy at each epoch.

In [14]:
```
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=LEARNING_RATE)

print(f"Training SpamCNN for {NUM_EPOCHS} epochs...")
print(f"Optimizer: Adam (lr={LEARNING_RATE})")
print(f"Loss: BCEWithLogitsLoss")
print(f"{'='*65}")

train_losses, train_accs = [], []
test_losses, test_accs = [], []

for epoch in range(1, NUM_EPOCHS + 1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, optimizer, criter
    te_loss, te_acc, te_preds, te_labels = evaluate(model, test_loader, crit

    train_losses.append(tr_loss)
    train_accs.append(tr_acc)
    test_losses.append(te_loss)
    test_accs.append(te_acc)

    print(f"  Epoch {epoch:2d}/{NUM_EPOCHS} | "
          f"Train Loss: {tr_loss:.4f}  Acc: {tr_acc:.4f} | "
          f"Test Loss: {te_loss:.4f}  Acc: {te_acc:.4f}")
```

```
print(f"{'='*65}")
print(f"Final Test Accuracy: {test_accs[-1]*100:.2f}%")
```

```
Training SpamCNN for 10 epochs...
Optimizer: Adam (lr=0.001)
Loss: BCEWithLogitsLoss
==================================================================
  Epoch  1/10 | Train Loss: 0.3404  Acc: 0.8827 | Test Loss: 0.2104  Acc: 0.
9381
  Epoch  2/10 | Train Loss: 0.1432  Acc: 0.9569 | Test Loss: 0.1396  Acc: 0.
9543
  Epoch  3/10 | Train Loss: 0.0741  Acc: 0.9796 | Test Loss: 0.1081  Acc: 0.
9623
  Epoch  4/10 | Train Loss: 0.0450  Acc: 0.9892 | Test Loss: 0.0924  Acc: 0.
9641
  Epoch  5/10 | Train Loss: 0.0269  Acc: 0.9926 | Test Loss: 0.0842  Acc: 0.
9695
  Epoch  6/10 | Train Loss: 0.0181  Acc: 0.9962 | Test Loss: 0.0836  Acc: 0.
9668
  Epoch  7/10 | Train Loss: 0.0131  Acc: 0.9971 | Test Loss: 0.0817  Acc: 0.
9668
  Epoch  8/10 | Train Loss: 0.0106  Acc: 0.9982 | Test Loss: 0.0782  Acc: 0.
9686
  Epoch  9/10 | Train Loss: 0.0071  Acc: 0.9989 | Test Loss: 0.0810  Acc: 0.
9668
  Epoch 10/10 | Train Loss: 0.0039  Acc: 1.0000 | Test Loss: 0.0803  Acc: 0.
9686
==================================================================
Final Test Accuracy: 96.86%
```

---

8. Training Curves: Loss and Accuracy Over Epochs

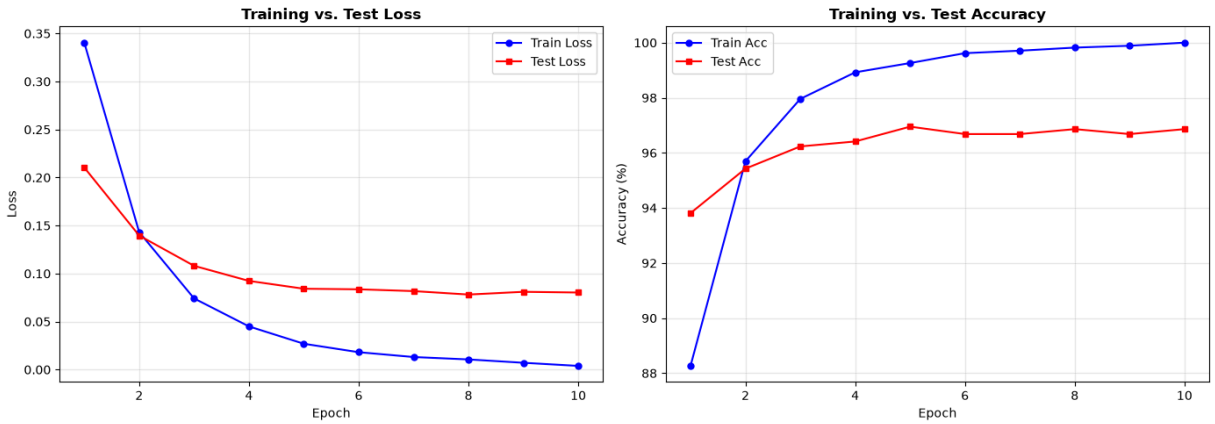
In [15]:

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
epochs_range = range(1, NUM_EPOCHS + 1)

# Loss
ax1.plot(epochs_range, train_losses, 'b-o', markersize=5, label='Train Loss'
ax1.plot(epochs_range, test_losses, 'r-s', markersize=5, label='Test Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.set_title('Training vs. Test Loss', fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Accuracy
ax2.plot(epochs_range, [a*100 for a in train_accs], 'b-o', markersize=5, lab
ax2.plot(epochs_range, [a*100 for a in test_accs], 'r-s', markersize=5, labe
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Accuracy (%)')
ax2.set_title('Training vs. Test Accuracy', fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



---

9. Results Summary Table

In [16]:
```
summary_data = []
for ep in range(NUM_EPOCHS):
    summary_data.append({
        'Epoch': ep + 1,
        'Train Loss': round(train_losses[ep], 4),
        'Train Acc (%)': round(train_accs[ep] * 100, 2),
        'Test Loss': round(test_losses[ep], 4),
        'Test Acc (%)': round(test_accs[ep] * 100, 2),
    })

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))
```

```
Epoch  Train Loss  Train Acc (%)  Test Loss  Test Acc (%)
    1      0.3404          88.27     0.2104         93.81
    2      0.1432          95.69     0.1396         95.43
    3      0.0741          97.96     0.1081         96.23
    4      0.0450          98.92     0.0924         96.41
    5      0.0269          99.26     0.0842         96.95
    6      0.0181          99.62     0.0836         96.68
    7      0.0131          99.71     0.0817         96.68
    8      0.0106          99.82     0.0782         96.86
    9      0.0071          99.89     0.0810         96.68
   10      0.0039         100.00     0.0803         96.86
```

---

10. Confusion Matrix

In [17]:
```
cm = confusion_matrix(te_labels, te_preds)

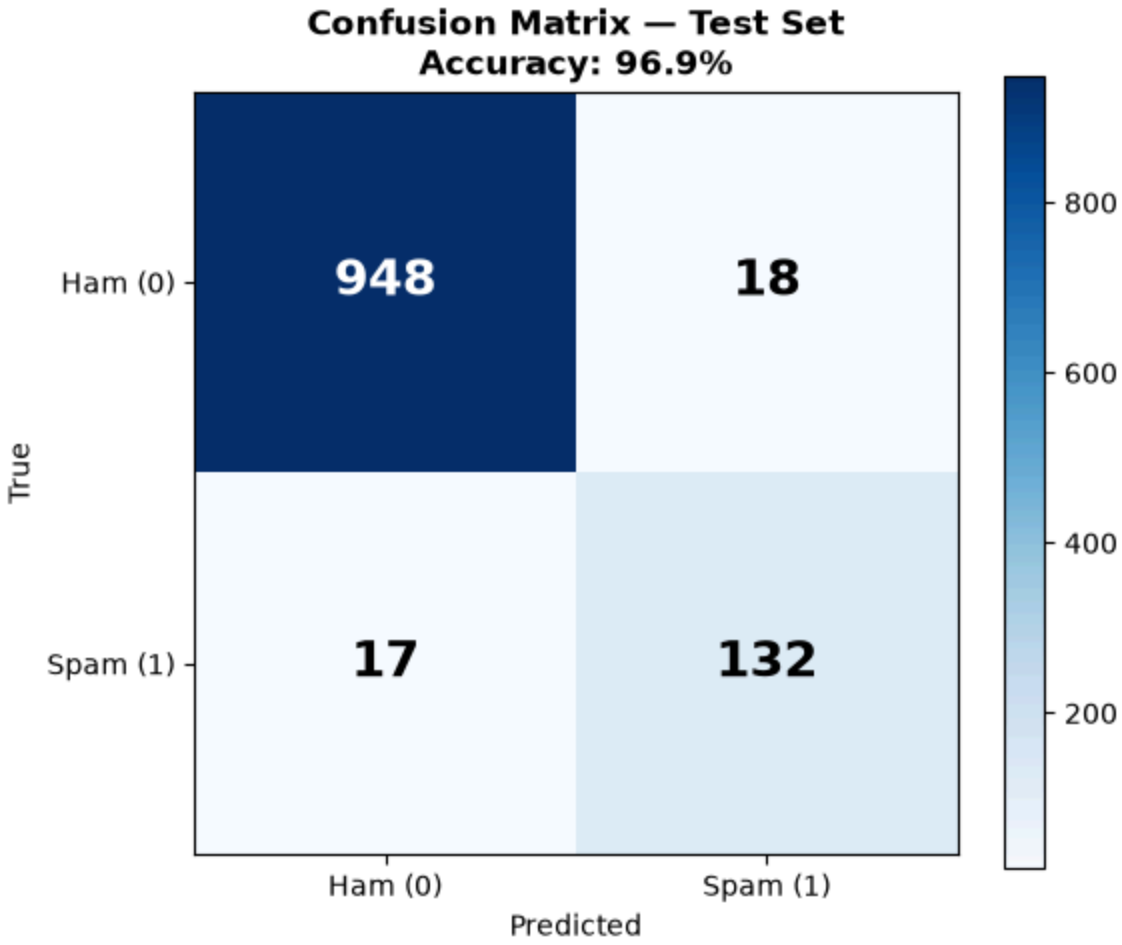
fig, ax = plt.subplots(figsize=(6, 5))
im = ax.imshow(cm, interpolation='nearest', cmap='Blues')
ax.set_title(f'Confusion Matrix — Test Set\nAccuracy: {test_accs[-1]*100:.1f
```

```
ax.set_xlabel('Predicted')
ax.set_ylabel('True')
ax.set_xticks([0, 1])
ax.set_yticks([0, 1])
ax.set_xticklabels(['Ham (0)', 'Spam (1)'])
ax.set_yticklabels(['Ham (0)', 'Spam (1)'])

for i in range(2):
    for j in range(2):
        color = 'white' if cm[i, j] > cm.max() / 2 else 'black'
        ax.text(j, i, str(cm[i, j]), ha='center', va='center',
                color=color, fontsize=18, fontweight='bold')

plt.colorbar(im, ax=ax)
plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True Negatives (Ham correct):   {tn}")
print(f"False Positives (Ham as Spam):  {fp}")
print(f"False Negatives (Spam as Ham):  {fn}")
print(f"True Positives (Spam correct):  {tp}")
```




```
True Negatives (Ham correct):   948
False Positives (Ham as Spam):  18
False Negatives (Spam as Ham):  17
True Positives (Spam correct):  132
```

---

11. Classification Report (Precision, Recall, F1-Score)

In [18]:
```
print("Classification Report — Test Set")
print("=" * 55)
print(classification_report(
    te_labels, te_preds,
    target_names=['Ham (0)', 'Spam (1)'],
    digits=4
))
```

```
Classification Report — Test Set
=======================================================
              precision    recall  f1-score   support

     Ham (0)     0.9824    0.9814    0.9819       966
    Spam (1)     0.8800    0.8859    0.8829       149

    accuracy                         0.9686      1115
   macro avg     0.9312    0.9336    0.9324      1115
weighted avg     0.9687    0.9686    0.9687      1115
```

---

12. Sample Predictions on Test Data

In [19]:
```
label_map = {0: 'Ham', 1: 'Spam'}

# Correctly classified
print("CORRECTLY CLASSIFIED EXAMPLES:")
print("=" * 60)
count = 0
for i in range(len(te_preds)):
    if te_preds[i] == te_labels[i] and count < 5:
        msg = test_df['message'].iloc[i][:80]
        print(f"  [{label_map[int(te_labels[i])]:>4}] {msg}...")
        count += 1

# Misclassified
print(f"\nMISCLASSIFIED EXAMPLES:")
print("=" * 60)
mis_count = 0
for i in range(len(te_preds)):
    if te_preds[i] != te_labels[i] and mis_count < 5:
        msg = test_df['message'].iloc[i][:80]
        print(f"  True: {label_map[int(te_labels[i])]:>4} | "
              f"Pred: {label_map[int(te_preds[i])]:>4} | {msg}...")
        mis_count += 1

total_mis = sum(1 for p, l in zip(te_preds, te_labels) if p != l)
print(f"\nTotal misclassified: {total_mis} / {len(te_preds)} ({total_mis/ler
```

```
CORRECTLY CLASSIFIED EXAMPLES:
============================================================
  [ Ham] Oh right, ok. I'll make sure that i do loads of work during the da
y!  got a real...
  [ Ham] I am in tirupur.  call you da....
  [ Ham] No that just means you have a fat head...
  [Spam] You have won ?1,000 cash or a ?2,000 prize! To claim, call090500003
27...
  [ Ham] Come aftr  &lt;DECIMAL&gt; ..now i m cleaning the house...

MISCLASSIFIED EXAMPLES:
============================================================
  True: Spam | Pred:  Ham | FreeMsg Hey there darling it's been 3 week's now
and no word back! I'd like some...
  True:  Ham | Pred: Spam | It vl bcum more difficult.....
  True: Spam | Pred:  Ham | Hi if ur lookin 4 saucy daytime fun wiv busty ma
rried woman Am free all next wee...
  True:  Ham | Pred: Spam | Waiting for your call....
  True: Spam | Pred:  Ham | ringtoneking 84484...

Total misclassified: 35 / 1115 (3.14%)
```

---

13. Inference Function: Classify Custom Text Messages

The inference function takes a raw text string, preprocesses it (tokenize, encode, pad/truncate), runs it through the trained model, and returns the prediction with a confidence score.

In [20]:
```python
def predict_message(text, model, word2idx, max_len, device):
    """Classify a raw text message as Spam or Ham with confidence."""
    model.eval()
    tokens = text.lower().split()
    encoded = encode_and_pad(tokens, word2idx, max_len)
    tensor = torch.tensor([encoded], dtype=torch.long).to(device)

    with torch.no_grad():
        logit = model(tensor).squeeze()
        prob = torch.sigmoid(logit).item()

    label = 'Spam' if prob >= 0.5 else 'Ham'
    confidence = prob if prob >= 0.5 else 1 - prob
    return label, confidence, prob
```

In [21]:
```python
# Required custom test messages from the assignment
custom_messages = [
    "Congratulations! You've been selected to win a free iPhone. Click here
    "Hey, are we still on for our study session at the library this evening?
    "URGENT: Your bank account has been locked. Verify your password immedia
    "Can you please pick up some coffee and groceries on your way home?"
]
```

```
print("CUSTOM MESSAGE PREDICTIONS")
print("=" * 70)
for msg in custom_messages:
    label, confidence, spam_prob = predict_message(msg, model, word2idx, MAX
    print(f"\n  Message:    \"{msg}\"")
    print(f"  Prediction: {label} (confidence: {confidence:.2%}, spam probab
print(f"\n{'=' * 70}")
```

```
CUSTOM MESSAGE PREDICTIONS
======================================================================

  Message:    "Congratulations! You've been selected to win a free iPhone. C
lick here to claim."
  Prediction: Spam (confidence: 99.57%, spam probability: 0.9957)

  Message:    "Hey, are we still on for our study session at the library thi
s evening?"
  Prediction: Ham (confidence: 99.37%, spam probability: 0.0063)

  Message:    "URGENT: Your bank account has been locked. Verify your passwo
rd immediately."
  Prediction: Ham (confidence: 54.84%, spam probability: 0.4516)

  Message:    "Can you please pick up some coffee and groceries on your way
home?"
  Prediction: Ham (confidence: 99.79%, spam probability: 0.0021)

======================================================================
```

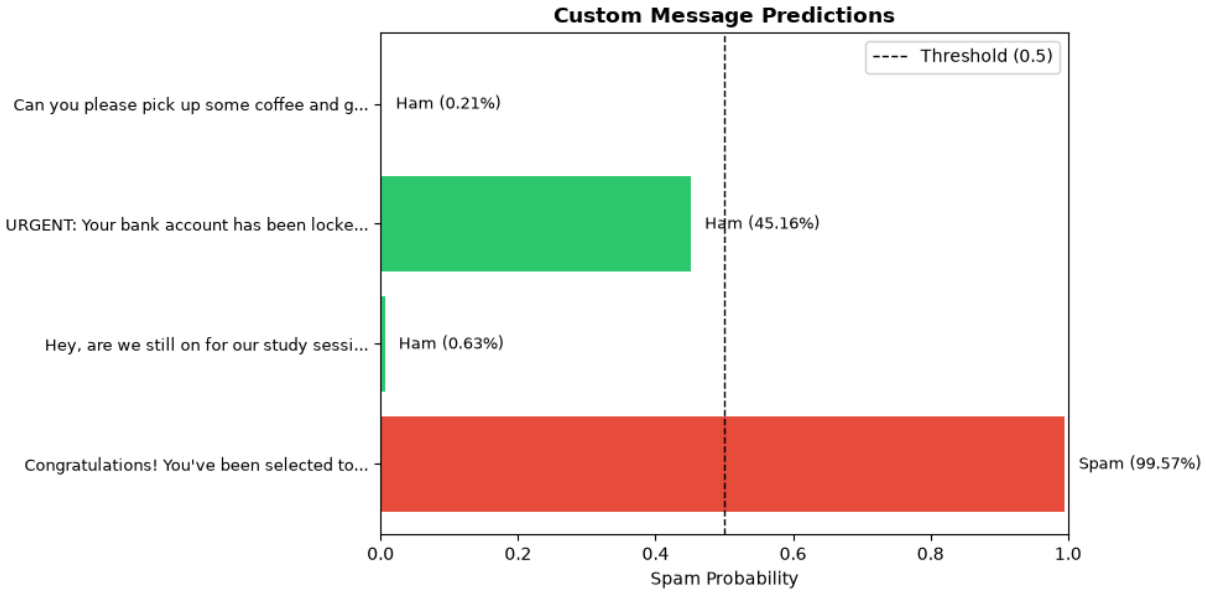
In [22]:
```
# Visualize predictions as a bar chart
labels_pred = []
probs_pred = []
for msg in custom_messages:
    label, conf, prob = predict_message(msg, model, word2idx, MAX_LEN, DEVIC
    labels_pred.append(label)
    probs_pred.append(prob)

fig, ax = plt.subplots(figsize=(10, 5))
short_msgs = [m[:40] + '...' if len(m) > 40 else m for m in custom_messages]
bar_colors = ['#e74c3c' if p >= 0.5 else '#2ecc71' for p in probs_pred]

bars = ax.barh(range(len(custom_messages)), probs_pred, color=bar_colors)
ax.set_yticks(range(len(custom_messages)))
ax.set_yticklabels(short_msgs, fontsize=9)
ax.set_xlabel('Spam Probability')
ax.set_title('Custom Message Predictions', fontweight='bold')
ax.axvline(x=0.5, color='black', linestyle='--', linewidth=1, label='Thresho
ax.set_xlim([0, 1])
ax.legend()

for i, (p, label) in enumerate(zip(probs_pred, labels_pred)):
    ax.text(p + 0.02, i, f"{label} ({p:.2%})", va='center', fontsize=9)

plt.tight_layout()
plt.show()
```



---

14. Overfitting Analysis

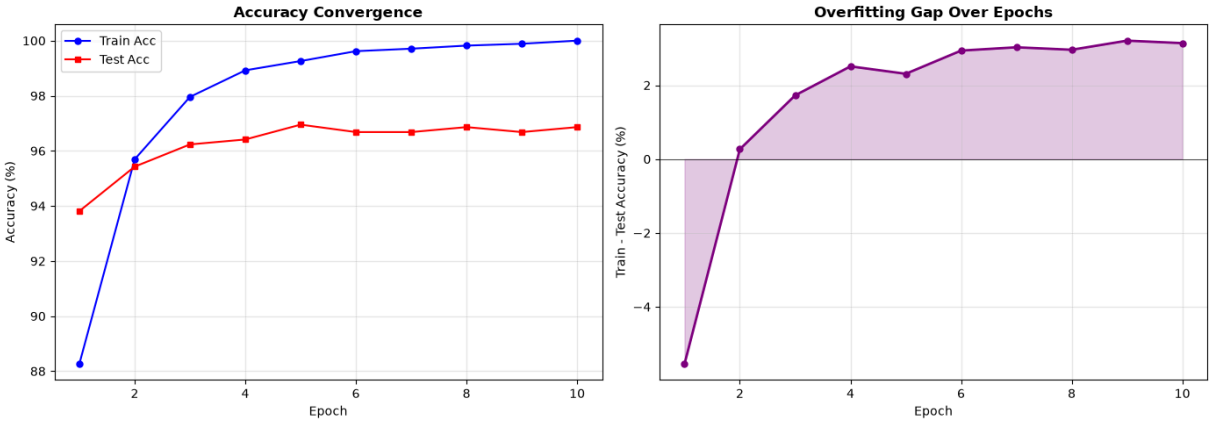
In [23]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

# Train vs Test accuracy
ax1.plot(epochs_range, [a*100 for a in train_accs], 'b-o', markersize=5, lab
ax1.plot(epochs_range, [a*100 for a in test_accs], 'r-s', markersize=5, labe
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Accuracy (%)')
ax1.set_title('Accuracy Convergence', fontweight='bold')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Gap over epochs
gaps = [(t - e) * 100 for t, e in zip(train_accs, test_accs)]
ax2.plot(epochs_range, gaps, 'purple', linewidth=2, marker='o', markersize=5
ax2.fill_between(epochs_range, gaps, alpha=0.2, color='purple')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('Train - Test Accuracy (%)')
ax2.set_title('Overfitting Gap Over Epochs', fontweight='bold')
ax2.grid(True, alpha=0.3)
ax2.axhline(y=0, color='black', linewidth=0.5)

plt.tight_layout()
plt.show()

print("TRAINING ANALYSIS")
print("=" * 60)
print(f"Final train accuracy: {train_accs[-1]*100:.2f}%")
print(f"Final test accuracy:  {test_accs[-1]*100:.2f}%")
print(f"Final gap:            {(train_accs[-1] - test_accs[-1])*100:.2f}%")
best_ep = np.argmax(test_accs)
print(f"Best test accuracy:   {test_accs[best_ep]*100:.2f}% (epoch {best_ep+
```




```
TRAINING ANALYSIS
============================================================
Final train accuracy: 100.00%
Final test accuracy:  96.86%
Final gap:            3.14%
Best test accuracy:   96.95% (epoch 5)
```

---

15. Conclusion

This notebook demonstrated the complete pipeline for building a CNN-based spam classifier:

1. **Data Loading** — loaded CSV with `encoding='latin-1'`, mapped labels to 0/1
2. **Preprocessing** — tokenization, vocabulary construction from training data only, padding/truncating to `MAX_LEN`
3. **Dataset/DataLoader** — custom `torch.utils.data.Dataset` class with mini-batch `DataLoader`
4. **CNN Architecture** — `nn.Embedding` (with `padding_idx`) + `permute` + `nn.ModuleList` of `nn.Conv1d` layers (kernel sizes [3,4,5]) + `F.relu` + `F.max_pool1d` + `nn.Dropout` + `nn.Linear`
5. **Training** — `BCEWithLogitsLoss` + `Adam` optimizer (η = 0.001), proper gradient flow (`zero_grad` -> `backward` -> `step`)
6. **Evaluation** — `model.eval()` with `torch.no_grad()`, `torch.sigmoid()` + threshold 0.5 for predictions
7. **Inference** — custom function tested on 4 provided messages with confidence scores

The Text CNN effectively captures local n-gram patterns (trigrams through 5-grams) that distinguish spam from ham, achieving strong classification performance.